UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

CMSC 12 — Fundamentals of Programming 2
Second Semester AY 2025-2026

MACHINE PROBLEM

EncanFrogger: The Adventures of Sang'gres

Technical Report

Submitted by:

Denise Jade Leguarda
BS in Computer Science
University of the Philippines Tacloban
City of Institution

Jessa Jasmine Parcero
BS in Computer Science
University of the Philippines Tacloban
Tacloban, Philippines

Jubiemyr Silvio
BS in Computer Science
University of the Philippines Tacloban
City of Institution

Lorenz Ed J. Ocampo
BS in Computer Science
University of the Philippines Tacloban
Tacloban, Philippines

Jezrel Marie Saño
BS in Computer Science
University of the Philippines Tacloban
Tacloban, Philippines

Instructor:

Samson Jr. D. Rollo
Bachelor of Science in Computer Science

20 May 2026

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

A. Rationale and Background

EncanFrogger: The Adventures of Sang'gres is an arcade-style game inspired by the classic Frogger and the GMA fantasy TV series Encantadia, set in a magical world of elemental kingdoms. In this game, players choose a Sang'gre as their character and navigate through dangerous paths filled with moving obstacles. Each Sang'gre features unique themes, paths, and challenges based on their elemental powers. The objective of the game is to carefully time movements, avoid collisions, and successfully reach the safe zone. It emphasizes quick decision-making, precise timing, and strategic movement as players progress through increasingly difficult levels. This arcade-style game is ideal for demonstrating core programming concepts due to its reliance on real-time interaction, collision systems, and state management. This makes the game suitable for applying OOP principles practically and engagingly.

B. Objectives

Our project aims to:

- Design and implement a 2D arcade-style game featuring Sang'gre characters with unique elemental abilities.
- Develop and integrate core game systems, including real-time player movement, collision detection using AABB, and level progression.
- Apply the 4 Pillars of Object-Oriented Programming(OOP) concepts, such as inheritance, encapsulation, polymorphism, and abstraction, in developing the game.
- Implement a game loop and event-driven input handling for smooth and responsive gameplay.
- Integrate graphics and animations using Java Swing/AWT for visual representation.
- Implement data persistence by saving and loading player data as well as leaderboard scores using file handling.
- Ensure smooth gameplay performance through efficient thread management.

C. Tools and Technologies

- Java (JDK 25): the primary programming language used to develop the game logic and systems due to its strong OOP support and built-in GUI libraries.
- Visual Studio Code (VS Code): a code editor used for writing, running, and debugging programs.
- GitHub: used for version control and collaboration among team members.
- Java Swing / AWT: lightweight framework suitable for creating the game window, rendering graphics, and handling user input and events.

- Figma: for designing user interface layouts and visual elements.
- Canva: for creating and editing backgrounds and other graphical assets.
- Piskel: for creating pixel-art sprites and simple animations
- Pixabay and Mixkit: sources of royalty-free sound effects used in the game.

D. Functional Requirements (Gameplay Dynamics)

Core Gameplay:

- **Movement:** The player character moves in four directions (up, down, left, right) in response to keyboard inputs: arrow keys or [W, A, S, D] . This will be grid-based to ensure consistent and accurate collision detection.
- **Obstacle Avoidance:** Moving obstacles traverse the screen in lanes. The player must cross these lanes without being hit.
- **Progression:** Levels increase in difficulty as the player advances. As the level progresses, the speed of the moving obstacles also increases.

Win / Loss Conditions:

- **Win:** The player successfully guides their Sang'gre to all safe zones within the time limit.
- **Loss:** The player collides with an obstacle or fails to complete the level before the timer expires. The player has three lives per session.
- A visual or audio feedback system will notify the player upon winning or losing a life.

Data Persistence:

- Upon game over, the player is prompted to enter 3-letter initials to claim their spot on the leaderboard, which is saved locally to a high scores file across sessions.

E. Methodology

The project will be developed as a 2D arcade-style game using Java (JDK 25) with Swing/AWT for rendering and user interaction. An incremental development approach will be followed, starting with a basic playable prototype and gradually adding features such as collision detection, character abilities, and data persistence. The project will be collaboratively managed using GitHub, allowing efficient integration of components.

Game assets such as Sang'gre characters, obstacles, and backgrounds will be designed using Figma, Canva, and Piskel, while sound effects will be sourced from Pixabay,

Mixkit, and other royalty-free libraries.

The system will be designed using the four pillars of Object-Oriented Programming (OOP):

- **Encapsulation:** Each class will contain its own data (position, speed, state) and methods, ensuring that internal details are protected and managed within the class.
- **Inheritance:** A base GameObject class will be created to hold common attributes shared by all game objects. Player will extend this class, and each Sang'gre (Terra, Flamara, Deia, and Adamus) will further extend Player, inheriting shared behaviors while implementing their own unique abilities.
- **Abstraction:** Complex systems such as the game loop, collision detection, and rendering will be simplified into manageable classes, allowing the team to focus on functionality without exposing unnecessary details.

The game loop will be managed across the following concurrent threads:

1. **GameLogicThread:** Updates player movement and object positions, then detects collisions using Axis-Aligned Bounding Box (AABB).
2. **RenderThread:** repaints updated frames to the screen.

User input will be handled through keyboard event listeners, allowing smooth control of the Sang'gre. A file handling system will be implemented to save and load player progress and leaderboard scores, ensuring persistence across sessions.

F. Results

The project successfully implemented the complete gameplay flow and user interface systems for *Encanfrogger: The Adventures of Sang'gres*. The game adapts the classic grid-based mechanics of Frogger while integrating themes, characters, and environments inspired by Encantadia.

Title Page

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

The game begins with a title page introducing the concept of navigating through dangerous environments while avoiding moving obstacles and crossing moving platforms safely.

Main Menu

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

Figure 1— The Main Menu was successfully implemented with three functional navigation buttons:

- **START** – directs the player to the Initials screen.
- **MENU** – opens the instructional guide containing gameplay rules and controls.
- **EXIT** – closes the application.

Initials / Registration Screen

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

Figure 2 — A player registration system was implemented where users can:

- Enter a username through a text field.
- Access the leaderboard through the Leaderboard button.
- Proceed to character selection using the Enter key.

Menu / How To Play

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

Figure 3— An instructional screen was implemented to display:

- Short game description
- Keyboard controls:
    - Up Arrow – move up
    - Down Arrow – move down
    - Left Arrow – move left
    - Right Arrow – move right
- Basic win and loss conditions

Character Selection Screen

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

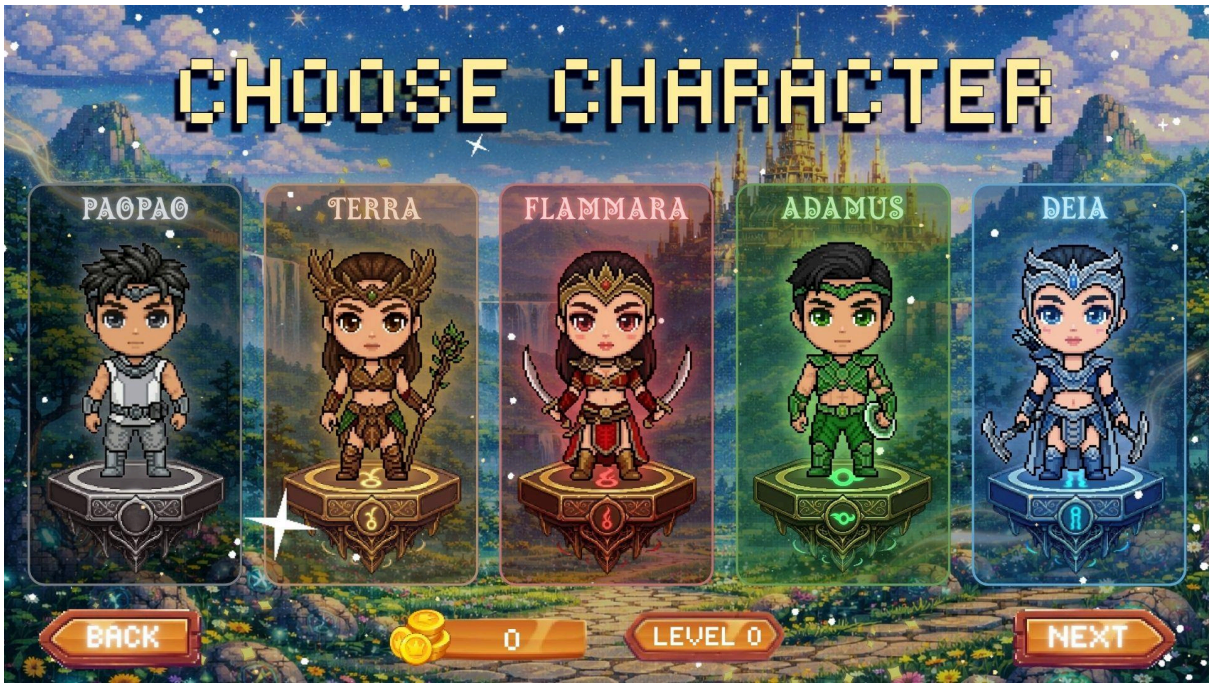

Figure 4— The game includes a functional character selection system featuring:

- Paopao
- Terra
- Flammara
- Adamus
- Deia

Additional UI elements implemented:

- Coin and level display
- Back button
- Next button with validation requiring character selection before proceeding

Map / Kingdom Selection Screen

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

Figure 5— Five playable themed kingdoms were successfully implemented:

- Lireo
- Hathoria
- Adamya
- Sapiro
- Mineave

The screen also includes:

- Coin and level display
- Back button
- Next button requiring map selection before gameplay starts

Gameplay Maps and Obstacles

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

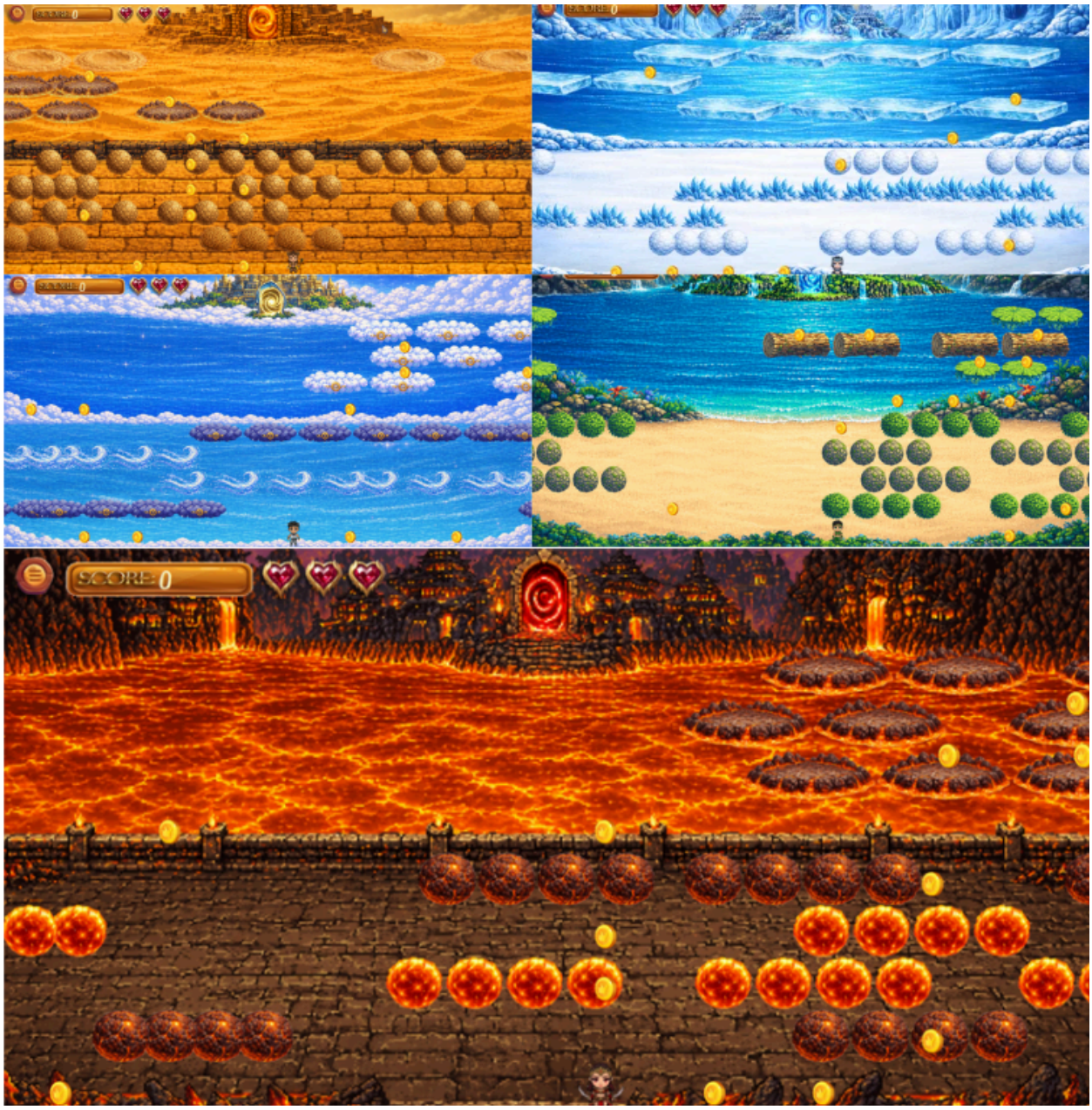

Figure 6— Each kingdom contains unique environmental themes, moving obstacles, and platform mechanics:

- **Sapiro** – desert environment with tumbleweeds, boulders, rock islands, and quicksand.
- **Hathoria** – lava-themed environment with fiery obstacles and volcanic rock platforms.
- **Mineave** – ice-themed environment with snowballs, ice crystals, and moving ice floes.
- **Adamya** – tropical coastal environment with shrubbery obstacles, logs, and lily pads.

- **Lireo** – sky-themed environment with storm clouds, wind gusts, and floating cloud platforms.

All maps include:

- Grid-based movement
- Obstacle avoidance mechanics
- Platform traversal sections
- Goal areas and win conditions
- HUD elements such as score display, heart lives, and menu button

Level Cleared Screen

Figure 7— A level completion screen was implemented displaying:

- Session statistics
- Level completion feedback
- Click-to-continue functionality for progression

Leaderboard Screen

Figure 8— A leaderboard system was successfully implemented to display:

- Highest-ranking player sessions
- Stored high scores
- Coin totals

The screen also contains:

- BACK button returning to Main Menu
- PLAY AGAIN button returning to the Initials screen

Game Over Screen

UNIVERSITY OF THE PHILIPPINES
Tacloban College
Division of Natural Sciences and Mathematics
Bachelor of Science in Computer Science

Figure 9— A Game Over screen was implemented when all three player lives are lost. The screen contains:

- "GAME OVER" display
- "PLAY AGAIN?" prompt
- YES button leading back to the Initials screen
- NO button leading to the Leaderboard screen

G. Problems/Bugs

- Previously, unreferenced objects were retained indefinitely, causing a continuous upward trajectory in heap consumption that ultimately led to system crashes. Following the optimization, memory utilization now exhibits a stable, cyclical pattern. While heap usage increases during active gameplay, it is immediately reclaimed and returned to the baseline once a garbage collection cycle executes.
- Verification and Testing - This behavioral correction was validated using VisualVM. By manually invoking the GC during profiling sessions, we observed the heap allocation immediately drop back to the initial baseline. This confirms that the application is correctly releasing object references. The upward slope observed in the telemetry graphs no longer represents a memory leak, rather, it reflects standard, deferred JVM cleanup behavior. Profiling confirms that no illegitimate objects are being retained in the heap, and all transient data is now fully eligible for collection.

- Configuration and Risk Mitigation - To guarantee a predictable memory footprint for the end-user, we have configured the executable JAR with a maximum heap restriction of 3GB (-Xmx3GB).
- This constraint is a proactive resource management strategy, not a performance limitation. Setting this threshold forces the JVM to optimize its cleanup intervals, rather than allowing deferred data to accumulate indefinitely, the JVM will automatically initiate a GC cycle as allocation approaches the 3GB ceiling. This boundary ensures the application remains highly stable and performant within standard hardware limits, without compromising visual quality, asset rendering, or frame rates.